

Base: A Board-Level Software ISA with Build-Time Dependent Type Checking

<https://github.com/unnamedlambda/base>

Abstract

A modern system board often contains many hardware units, each with its own ISA: a CPU, a GPU, and specialized accelerators. Programming them correctly requires reasoning about value-dependent properties such as memory layouts, buffer offsets, and cross-device data flow. Dependent type theory can express these constraints, but languages with sufficiently expressive type systems often incur managed run-time overhead (for example, garbage collection, reference-count traffic, boxing, and other run-time support costs). We therefore want dependent type checking to happen at build time and to execute code that maps well onto modern hardware, as modern systems languages commonly do (e.g., C and C++).

Base solves this with a clean separation. At build time, Lean 4 [1] with dependent types produces a serializable execution artifact containing Cranelift [2] IR text, an initial memory layout, and one or more compiled-function entry points. At run time, a Rust engine JIT-compiles the IR and calls the selected entry point. No type checker, no interpreter, no garbage collector runs at run time—just compiled machine code calling hardware through FFI primitives. The data artifact is the only thing that crosses from the verified world to the executable world, and it contains no closures, callbacks, or run-time artifacts from Lean.

This supports a modern heterogeneous system with multiple ISAs following different conventions. The ISAs do not need to be designed to work intuitively together—they can be awkward, optimized purely for silicon, because the dependent type system absorbs their complexity at build time. Base is evaluated on CPU, GPU, and heterogeneous workloads. On the evaluated workloads, it is competitive with idiomatic Rust and native GPU frameworks.

1 Introduction

Board-level software must coordinate multiple ISAs on one system board (CPU, GPU, tensor/crypto engines, and other accelerators). Doing this correctly requires reasoning about shared memory layouts, buffer offsets, dispatch ordering, and data flow between devices.

These are value-dependent invariants. A buffer offset must be less than the memory size. A GPU dispatch must happen after initialization. A data layout must be consistent between the CPU view and the GPU view. Dependent types allow types to depend on values, so bounds, layout relationships, and sequencing conditions can participate directly in artifact construction and checking. That is useful here because offsets, sizes, and dispatch structure all depend on concrete build-time values.

Dependent type checking requires substantial compile-time machinery. In Lean 4, checking depends on term reduction/normalization. None of this proof-assistant machinery should be in the run-time path.

Base separates the problem in two:

1. **Build time** (Lean 4): Verify program structure with dependent types. Output a serializable data artifact containing Cranelift IR text, an initial memory layout, and named entry points into the generated CLIF, with no Lean run-time artifacts.
2. **Run time** (Rust engine): JIT-compile the IR via Cranelift. Execute selected CLIF functions against shared memory, calling hardware through typed FFI primitives.

The data artifact is inert: bytes, offsets, IR text, integer tags. It is the only value that crosses from the verified world to the executable world.

A consequence: ISAs targeted by this system do not need to be programmer-friendly. If the specification language handles complexity, hardware can be optimized purely for silicon efficiency—what might be called *awkward ISAs*. Because no human or general-purpose compiler targets the IR directly, instruction sets can be irregular—non-power-of-two register files, application-specific operations, unusual word sizes—and the Lean specification provides verified transformations to each device on the bus.

2 Architecture

Figure 1 shows the system. Lean constructs the data artifact at build time; the Rust engine consumes it at run time.

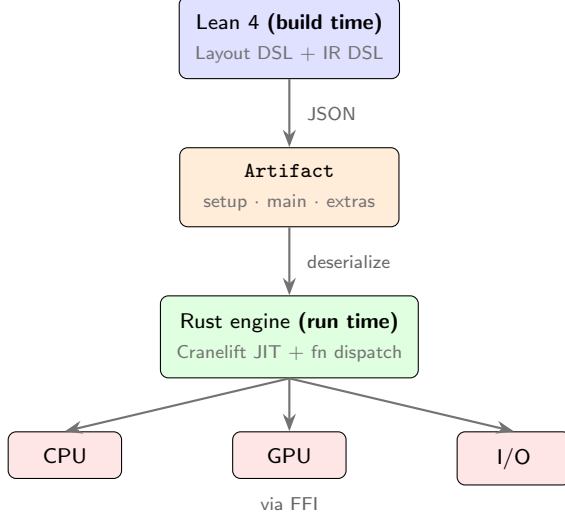


Figure 1: Lean builds the data artifact at build time; the Rust engine consumes it at run time and dispatches via FFI.

The build-time phase runs once per program revision. Lean 4 type-checks the specification, emits a flat memory layout and Cranelift IR text, and serializes the resulting **Artifact** as JSON. Rust `build.rs` then validates that JSON against the Rust data types, re-serializes it to bincode, and embeds it in the executable via `include_bytes!`. At run time the engine deserializes the embedded artifact, JIT-compiles the IR once during `Base::new`, and executes the `main` algorithm or a named algorithm from `extras` by calling its compiled function index.

3 The Data Artifact

A Base program is a single **Artifact**, serialized as JSON:

```

pub struct Artifact {
  pub setup: Setup,
  pub main: Algorithm,
  pub extras: HashMap<String, Algorithm>,
}
pub struct Setup {
  pub cranelift_ir: String,
  pub memory_size: usize,
  pub io_offsets: IoOffsets,
  pub initial_memory: Vec<u8>,
}
pub struct IoOffsets {
  pub data_ptr: usize,
  pub data_len: usize,

```

```

  pub out_ptr: usize,
  pub out_len: usize,
}
pub struct Algorithm {
  pub fn_idx: u32,
  pub output: Vec<OutputBatchSchema>,
}

```

Setup contains the module-level Cranelift IR text, memory size, runtime input/output slots, and byte-level initial memory generated by Lean. **Algorithm** is an entry point into that compiled module: `fn_idx` selects one JIT-compiled function, and `output` describes optional Arrow record batches to read from shared memory after the call. A single-artifact program uses `main`; multi-stage flows put additional entry points in `extras`, so all stages share one memory image and one Cranelift compilation.

Orchestration is represented either inside generated CLIF functions or by explicit calls to different **Algorithm** entry points by the host application.

3.1 Shared Memory

All state lives in a single pinned byte array. When a **Base** struct is created, the array is initialized from `initial_memory`, which holds program data, string constants, and scratch buffers produced by Lean. Every JIT-compiled function receives one argument: a pointer to byte 0 of this array.

For caller-provided input and output buffers, `Setup.io_offsets` specifies where the engine writes transient pointer slots before each `execute()` call (Table 1). These offsets are part of the artifact rather than a hard-coded Rust header.

Table 1: Runtime input/output slots specified by `IoOffsets`.

Field	Value written before execution
<code>data_ptr</code>	pointer to caller input bytes
<code>data_len</code>	caller input length
<code>out_ptr</code>	pointer to caller output buffer
<code>out_len</code>	caller output length

CLIF code reads `data_ptr/data_len` to access caller input and `out_ptr/out_len` to write results—zero copies. GPU, CUDA, hash-table, network, and database contexts are managed through FFI calls that initialize and pass context handles explicitly, not through a single serialized context offset in the artifact.

3.2 Execution

```
// One-shot
let artifact = Artifact::from_bytes(ARTIFACT_BINARY);
let results = run(artifact.setup, artifact.main?);

// Compile once, execute many
let artifact = Artifact::from_bytes(ARTIFACT_BINARY);
let mut base = Base::new(artifact.setup?);
base.execute(&artifact.main, &data?);

// Named multi-stage entry points
let prep = &artifact.extras["prep"];
let infer = &artifact.extras["infer"];
base.execute(prepare, &input?);

// Zero-copy output
base.execute_into(infer, &in, &mut out?);
```

After JIT compilation, `execute()` is a hot path: callers pass data by reference, the engine writes the caller pointers to the configured `IoOffsets`, and then calls `algorithm.fn_idx` in the compiled module. CLIF code reads caller data directly.

3.3 FFI: From Rust to C ABI to Cranelift

Hardware access follows a three-step chain. First, a Rust function wraps a driver library (wgpu [4], CUDA, LMDB [5], TCP) behind `extern "C"` linkage:

```
unsafe extern "C" fn cl_file_read_to_ptr(
    path_ptr: *const u8, // filename pointer
    dst_ptr: *mut u8, // destination pointer
    file_offset: i64, // seek position
    size: i64, // bytes to read
) -> i64 {
    let filename = read_cstr_ptr(path_ptr);
    let mut file = match fs::File::open(&filename) {
        Ok(f) => f, Err(_) => return -1,
    };
    // ... read into dst_ptr, return count
}
```

Second, the engine registers this function pointer with the Cranelift JIT builder under its symbol name:

```
builder.symbol("cl_file_read_to_ptr",
    cl_file_read_to_ptr as *const u8);
```

Third, Lean can target the same symbol in two equivalent ways. Small benchmarks in this repository often emit the raw CLIF directly:

```
sig0 = (i64, i64, i64, i64) -> i64 system_v
fn0 = %cl_file_read_to_ptr sig0
```

```
block4:
    v3 = iconst.i64 256
    v4 = iadd v0, v3 ; filename pointer
    v5 = iconst.i64 8192
    v6 = iadd v0, v5 ; destination pointer
    v7 = iconst.i64 0
    v8 = iconst.i64 4096
    v9 = call fn0(v4, v6, v7, v8)
```

Larger applications use the IR builder, which produces the same external declaration while returning a typed handle:

```
def declareFileRead : IRBuilder FnRef :=
    declareFFI "cl_file_read_to_ptr"
        [.i64, .i64, .i64, .i64] (some .i64)
```

In both cases the generated `cranelift_ir` contains the same external symbol reference. At JIT time, Cranelift resolves `%cl_file_read_to_ptr` to the registered pointer—a direct native call, no extra dispatch layer. The same three-step pattern applies to all registered FFI symbols (Table 2).

Table 2: FFI primitive categories (88 registered symbols total).

Category	n	Examples
Hash table	8	init, insert, lookup, increment
GPU (wgpu)	9	init, ptr upload, dispatch
CUDA/cuBLAS	39	streams, graphs, PTX launch, GEMM
File/math/I/O	9	file read/write, trig, stdin/stdout
Networking	8	listen, connect, send
Database	10	LMDB transactions
Threading	5	spawn, join, direct call

Adding a new device means: (1) write `extern "C"` Rust wrappers, (2) register symbols in the JIT builder, (3) add a `declareFFI` bundle in Lean. The artifact format and function-index dispatch model do not change.

4 Specification in Lean

The goal of the Lean build-time phase is always the same: produce a valid `Artifact`. Figure 2 shows the end-to-end picture. What Lean emits—Cranelift IR text, an initial memory layout, and `main/extras` entry points—is fixed by that type. Any Lean code that type-checks and produces such an artifact is a legitimate front end. The DSLs described below are one convenient approach used in this repository, built

from standard functional programming techniques in Lean; they are not the only option, and users may write whatever transformations suit their domain.

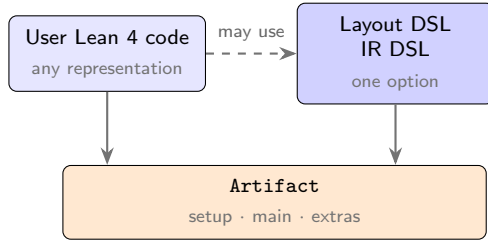


Figure 2: Any Lean code that produces the data artifact is valid. In this repository, both raw CLIF string emitters and the monadic DSL are used; both are just front ends to the same artifact.

4.1 The Transformation

Figure 3 shows what the DSLs do at a high level. On the left: typed Lean values—field declarations with their sizes and alignments, and SSA operations with their operands. On the right: the two outputs that populate the data artifact. The transformation is purely functional; standard stateful-computation patterns in Lean thread a byte cursor through field declarations and an instruction list through IR construction. The key property is that Lean computes offsets, sizes, and SSA value IDs during artifact construction, which lets the surrounding types and builders impose constraints before any code runs.

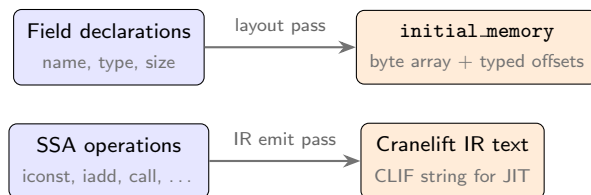


Figure 3: The two DSL passes: field declarations map to a typed byte layout; SSA operations map to Cranelift IR text. Both are pure Lean functions.

4.2 Memory Layout

The layout stage produces the concrete `initial_memory` byte array, the total `memory_size`, and typed references to important regions. In the current applications those outputs include string constants, headers, descriptors, scratch buffers, and runtime-visible input/output regions.

What matters is that later CLIF generation refers to these generated locations by named Lean values rather than manually duplicated offsets. Lean computes the byte positions and serialized initial contents at build

time, so the artifact carries both a concrete memory image and the locations that the generated code will read, write, and pass to FFI calls.

5 Example: GPU Path Tracer

To illustrate the full pipeline, this section traces a complete application: a 4096×4096 Cornell box path tracer with 5-bounce ray tracing. The entire specification—WGSL shader, Cranelift IR, BMP header, and memory layout—is a single Lean file.

5.1 What Lean Produces

The Lean file constructs the full **Artifact**:

Memory layout. The `initial_memory` byte array contains, at known offsets: the output filename string, a 54-byte BMP header (with width, height, and pixel format pre-filled), the WGSL shader source (~ 16 KB of text), a GPU binding descriptor, and a zero-filled pixel buffer large enough for $4096 \times 4096 \times 4$ bytes (~ 67 MB). Every offset is a named Lean constant, checked against the total `memory_size`.

Cranelift IR. The IR builder emits a single function (`u0:0`) that orchestrates the GPU through seven FFI calls:

```

def clifIrSource : String := buildProgram do
  let gpu <- declareGpuFFI
  let fnWr <- declareFileWrite
  let ptr <- entryBlock
  -- GPU lifecycle: init, buffer, shader,
  -- dispatch, download, cleanup
  callVoid gpu.fnInit [ptr]
  let sz <- iconst64 pixelBytes
  let buf <- call gpu.fnCreateBuffer [ptr, sz]
  let pipe <- call gpu.fnCreatePipeline
    [ptr, shaderOff, bindOff, one32]
  let _ <- call gpu.fnDispatch
    [ptr, pipe, wgX, wgY, one32]
  let _ <- call gpu.fnDownload
    [ptr, buf, pixelOff, sz]
  callVoid gpu.fnCleanup [ptr]
  -- Write BMP: header (54B) + pixels
  let total <- iconst64 (54 + pixelBytes)
  let _ <- writeFile0 ptr fnWr
    filenameOff headerOff total
  ret
  
```

Algorithm entry point. The artifact’s main algorithm selects that function:

```
def raytraceAlgorithm : Algorithm := {
  fn_idx := IR.mainFnIdx
  output := []
}
```

5.2 What the Engine Does

Figure 4 shows the run-time execution flow. The Rust side is three lines: deserialize, construct `Base`, call `execute`.

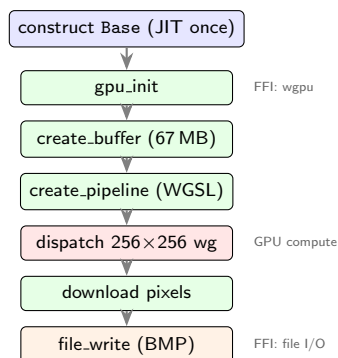


Figure 4: Path tracer execution: one CLIF function orchestrates the GPU through FFI calls. Total: ~735 ms.

The JIT-compiled native code calls `cl_gpu_init` to initialize a wgpu [4] context handle, creates a buffer and pipeline, dispatches 256×256 workgroups (each covering a 16×16 pixel tile), downloads the result into the pixel region of shared memory, and writes the BMP file. The entire GPU lifecycle—from shader compilation to file output—is driven by a single Cranelift function that Lean generated and type-checked.

5.3 What Lean Prevents

Without dependent types, common errors in this pipeline include: writing the shader string past its allocated region, using the wrong pixel buffer offset in the download call, or sizing the BMP header inconsistently with the pixel format. In Base, the Lean type system can rule out these classes of mistakes at build time: `shaderOff` is a typed field handle whose offset is computed by the layout builder, and using it with an incompatible size is a type error. In the generated path-tracer artifact, those checks eliminate the need for corresponding run-time offset validation in the engine.

6 Build Pipeline

Figure 5 shows how a Lean source becomes an executable.

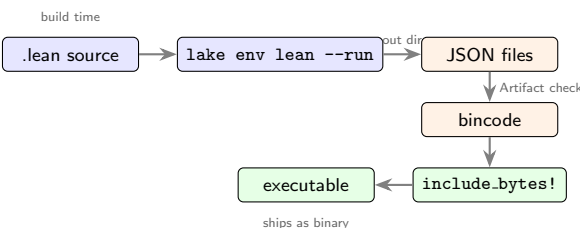


Figure 5: Build pipeline: Lean runs at `cargo` build time via `build.rs`. The data artifact is baked into the Rust binary.

`build.rs` first invokes `lake build` for the Lean module, then runs `lake env lean --run <file> <out-dir>`. The Lean program writes one JSON file per artifact into that output directory. The build script validates each file against the Rust `Artifact` type via `serde_json`, re-serializes it to bincode, and writes the result to `$OUT_DIR`. The Rust binary includes this blob via `include_bytes!` and deserializes it with `Artifact::from_bytes` at startup.

7 Evaluation

All benchmarks were run on the same machine: AMD Ryzen 5 5500 (6 cores / 12 threads), NVIDIA RTX 3060 (12 GB), 32 GB DDR4, Linux 6.14. Reported times are medians of 10 rounds from the current harness, and each row is checked for correctness against a reference implementation. The full suite is invoked by `./benchmarks/run.sh`, which builds the Python extension and Python-generated artifacts first, then delegates the Rust suite to `cargo run --release -p benchmarks`. The two suites report different comparisons: the Python suite compares Python libraries with PyBase, while the Rust suite compares native Rust, Burn/Rayon where applicable, and Base.

7.1 CPU

On larger file and string-processing workloads, Base is competitive with and often faster than the current Rust baselines: CSV 5M runs in 141.4 ms vs. 310.5 ms, JSON 500K in 9.7 ms vs. 15.9 ms, Regex 1M in 4.2 ms vs. 6.7 ms, and StrSearch 1M in 1.3 ms vs. 2.2 ms. Small inputs show the fixed cost of entering the engine and calling FFI. For workloads such as CSV and Regex, the generated CLIF uses explicit SIMD instructions such as `splat.i8x16`, `icmp`, and `vhigh_bits` rather than relying on auto-vectorization.

Table 3: Python-facing benchmarks. The Python column is Python, NumPy, Pandas, or PyTorch depending on the row; PyBase is the same Base engine called through the Python extension.

	Python	PyBase
CSV 1M	496.3 ms	26.4 ms
JSON 500K	312.8 ms	9.44 ms
Regex 1M	83.1 ms	4.83 ms
StrSearch 1M	3.73 ms	1.51 ms
WordCnt 1M	110.2 ms	37.4 ms
VecAdd 50M	43.5 ms	12.9 ms
Pandas RevAgg 10M	86.8 ms	7.88 ms
RMSNorm 4096	0.059 ms	0.012 ms

Table 4: Rust CPU file/string benchmarks from the current harness.

	Rust	Base
CSV 10K	0.3 ms	1.1 ms
CSV 1M	57.6 ms	27.7 ms
CSV 5M	310.5 ms	141.4 ms
JSON 1K	0.0 ms	0.8 ms
JSON 500K	15.9 ms	9.7 ms
Regex 10K	0.1 ms	0.9 ms
Regex 1M	6.7 ms	4.2 ms
StrSearch 1M	2.2 ms	1.3 ms
WordCnt 1M	32.9 ms	37.1 ms

Table 5: Numerical compute (4×-unrolled SIMD in Base).

	Rust	Burn	Base
MatMul 128	0.2 ms	0.1 ms	0.2 ms
MatMul 512	12.0 ms	1.1 ms	7.7 ms
MatMul 1024	98.9 ms	8.1 ms	59.0 ms
VecAdd 1M	0.7 ms	0.2 ms	0.1 ms
VecAdd 10M	7.2 ms	25.4 ms	2.4 ms
VecAdd 50M	36.0 ms	117.3 ms	12.6 ms
Sum 1M	0.7 ms	0.1 ms	0.1 ms
Sum 10M	7.1 ms	1.8 ms	1.5 ms
Sum 100M	71.4 ms	18.5 ms	16.3 ms

On numerical CPU workloads, Base about matches Rust overall and is substantially faster than Burn on simple element-wise operations: VecAdd 10M runs in 2.4 ms vs. 7.2 ms for Rust and 25.4 ms for Burn, and Sum 100M runs in 16.3 ms vs. 71.4 ms and 18.5 ms respectively. Burn still exceeds Base on matmul because it routes to specialized backend kernels. WordCount is the notable exception on the file-processing side: at 1M words Base is slower than Rust (37.1 ms vs. 32.9 ms), most plausibly because Base goes through external `ht_*` FFI calls that Cranelift cannot inline, whereas LLVM can optimize the Rust `HashMap` path more aggressively.

Table 6: GPU benchmarks (one-shot: upload + compute + download).

	Burn (wgpu)	Base
VecAdd 256K	4.2 ms	0.7 ms
VecAdd 500K	3.9 ms	1.2 ms
MatMul 256	3.0 ms	0.4 ms
MatMul 512	3.9 ms	1.2 ms
Reduce 256K	3.0 ms	0.4 ms
Reduce 512K	3.5 ms	0.5 ms
Reduce 896K	4.1 ms	0.8 ms

Table 7: GPU iterative (data stays on device).

	Raw wgpu	Burn	Base
1× 1M	3.9 ms	4.4 ms	1.7 ms
10× 1M	4.0 ms	5.0 ms	2.2 ms
100× 1M	8.5 ms	7.0 ms	6.7 ms
500× 1M	25.8 ms	22.5 ms	26.9 ms
1000× 1M	49.4 ms	41.3 ms	49.4 ms

Table 8: CUDA (direct PTX, no compile-time toolkit).

	Burn (cuda)	Base
SAXPY 262K	0.5 ms	0.5 ms
SAXPY 524K	0.9 ms	0.8 ms
SAXPY 1M	1.8 ms	1.7 ms

Table 9: Parallel histogram.

	Rust	Rayon	Base
Hist 1M $w=1$	1.1 ms	1.0 ms	1.6 ms
Hist 1M $w=4$	0.8 ms	0.9 ms	1.4 ms
Hist 10M $w=1$	10.8 ms	11.5 ms	10.8 ms
Hist 10M $w=4$	8.6 ms	8.4 ms	7.9 ms

7.2 GPU, CUDA, and Stress Tests

On one-shot GPU workloads, Base is consistently faster than Burn’s `wgpu` backend: VecAdd 256K runs in 0.7 ms vs. 4.2 ms, MatMul 512 in 1.2 ms vs. 3.9 ms, and Reduce 896K in 0.8 ms vs. 4.1 ms. Because both implementations use `wgpu` on the same hardware, this is primarily a difference in host-side runtime overhead rather than a different GPU path.

The iterative GPU-resident benchmark is more balanced. Base wins at 1× and 10× passes (1.7 ms and 2.2 ms), remains close at 100×, and by 500×–1000× passes Burn and raw `wgpu` catch up or pull ahead. That is consistent with a workload that becomes dominated by repeated kernel execution rather than orchestration. CUDA is also close: Base runs SAXPY 1M in 1.7 ms versus 1.8 ms for Burn(cuda).

Table 9 shows an additional CPU-side stress test. Base is close on parallel histogramming.

8 Conclusion

Base separates dependent-type verification (Lean 4, build time) from native execution (Rust engine, Cranelift JIT, run time) through a serializable **Artifact**. No type checker runs at run time. On the evaluated workloads, it is competitive with idiomatic Rust and native GPU frameworks.

The artifact is emitted as JSON at build time and embedded as bincode at run time, so the model extends naturally to board-level programming. Today the engine dispatches to CPU, and exposes FFI primitives for wgpu, CUDA, networking, file I/O, LMDB, and threading. Each device keeps its awkward ISA; Lean verifies cross-device constraints at build time, and the engine orchestrates at run time.

Any Lean code that produces a valid **Artifact** is a legitimate front end. The monadic DSLs presented here are one choice; users are free to build higher-level representations—algebraic IRs, domain-specific abstractions, or full compiler pipelines—and have dependent types flow through to the final algorithm emission. Layout errors, offset mismatches, and device-compatibility constraints can all be caught interactively as programmers construct objects, making the value of dependent types immediately visible rather than deferred to a separate verification step.

Base targets CPU and GPU backends at the driver-API level—Cranelift for native code, the CUDA driver API for PTX, and wgpu for portable GPU compute—and exposes them to Lean as FFI primitives. This means the optimization and orchestration logic that would otherwise live in a C++ compiler stack can instead be written in Lean, where a strong type system and proof assistant make it simpler to write correct transformations.

References

- [1] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Proc. CADE-28*, 2021.
- [2] Bytecode Alliance, “Cranelift code generator,” <https://cranelift.dev>, 2024.
- [3] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis and transformation,” in *Proc. CGO*, 2004.
- [4] gfx-rs contributors, “wgpu: Safe and portable GPU abstraction,” <https://wgpu.rs>, 2024.
- [5] H. Chu, “LMDB: Lightning memory-mapped database,” <http://www.lmdb.tech/doc/>, 2011.
- [6] Burn contributors, “Burn: A flexible and comprehensive deep learning framework in Rust,” <https://burn.dev>, 2024.